

# Universidad Don Bosco

Facultad de Ingeniería

Escuela de Computación

Desarrollo de Software Empresarial (G01T)

Ciclo 02/2026



## Proyecto de cátedra Fase 1: Desarrollo de página de comercio electrónico para emprendimiento “Chocolates SV”

Integrantes:

| Apellidos           | Nombres         | Carnét   |
|---------------------|-----------------|----------|
| Contreras Bojórquez | Melany Fabiola  | CB221855 |
| Escobar Valle       | Arnulfo Antonio | EV220288 |
| Díaz Carpio         | Ariel Ernesto   | DC221025 |
| Salguero Quezada    | Felipe Alberto  | SQ220190 |

Docente:

Mg. Emerson Francisco Cartagena

Antiguo Cuscatlán, 29 de septiembre del 2026

## Contenido

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 3.INTRODUCCION.....                                                    | 4  |
| 3.1 Contexto del Emprendimiento y Planteamiento del Problema .....     | 4  |
| 3.2 Propósito y Alcance del Sistema .....                              | 5  |
| 3.3 Objetivos (General y Específicos) .....                            | 5  |
| 4. Gestión y Metodología de Trabajo.....                               | 6  |
| 4.0.1 Links a los apartados del trabajo (Trello, GitHub, Figma) .....  | 6  |
| 4.1 Metodología Ágil (Scrum) .....                                     | 6  |
| 4.2 Asignación de Roles del Equipo .....                               | 9  |
| 4.3 Gestión de Tareas y Versionamiento .....                           | 10 |
| 5. Diseño UX/UI y Prototipado .....                                    | 11 |
| 5.1 Guía de Estilo Visual .....                                        | 11 |
| 5.2 Mockups Principales (Figma) .....                                  | 12 |
| 6. Estrategia de Solución y Arquitectura Técnica .....                 | 19 |
| 6.1. Arquitectura General del Sistema .....                            | 19 |
| 6.2 Patrón Arquitectónico del Backend: Arquitectura en N-Capas .....   | 21 |
| 6.2.2 Inyección de Dependencias y Modularidad (Extension Methods)..... | 22 |
| 6.3 Mapeo y Flujo de las APIs RESTful .....                            | 23 |
| 6.3.1 APIs del Modo Cliente (Sin Cuenta / Invitado) .....              | 23 |
| 6.3.2 APIs del Modo Administrador (Autenticación con “Identity”).....  | 24 |
| 6.4. Modelado de Datos .....                                           | 26 |
| 6.4.1 Diagrama Entidad-Relación.....                                   | 26 |
| 6.4.2 Diagrama de base de datos.....                                   | 27 |
| 6.4.3 Diccionario de base de datos .....                               | 27 |
| 6.4.4 Estructura en ChocolatesSV.Entities (Models y DTOs) .....        | 37 |
| 6.4.5 Mapeo de Objetos con AutoMapper(ChocolatesSV.BL.Profiles).....   | 37 |
| 6.4.6 Persistencia con Micro-ORM Dapper (ChocolatesSV.DAL) .....       | 37 |

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| 6.5. Diagramas UML de Comportamiento y Estructura.....                         | 38 |
| 6.5.1 Diagrama de Casos de Uso (Cliente vs Administrador).....                 | 38 |
| 6.5.2 Diagramas de Secuencia clave Proceso de Checkout / Compra.....           | 39 |
| 6.5.3 Diagrama de secuencia para el mantenimiento del catálogo .....           | 39 |
| 6.5.4 Diagrama de secuencia para rastreo de pedido de cliente (Invitado) ..... | 39 |
| 6.5. Diagrama de Clases del Dominio.....                                       | 40 |
| 6.6. Estrategia de Seguridad y Validaciones .....                              | 40 |
| 7. Stack Tecnológico y Herramientas .....                                      | 41 |
| 8. Licenciamiento .....                                                        | 42 |
| 9. Bibliografía .....                                                          | 42 |

Links para montar proyecto:

API (backend)- <https://github.com/arieldiazdev10/ChocolatesSV.API>

Administrador (frontend)- <https://github.com/arieldiazdev10/admin-chocolates>

Cliente (frontend)-- <https://github.com/KimiArnul/cliente-chocolates>

### 3.INTRODUCCION

El presente proyecto consiste en el desarrollo de una página web para el emprendimiento "**Chocolates SV**", orientada principalmente al uso desde computadora, pero completamente adaptable a dispositivos móviles y tabletas (para ambiente web, no como aplicación nativa).

El objetivo principal de la plataforma es digitalizar y optimizar la forma en que el emprendimiento presenta su catálogo de productos, gestiona promociones, procesa pedidos, administra sus operaciones y mantiene comunicación con sus clientes.

La plataforma estará dividida en dos modos principales de uso: **Modo Cliente**, orientado a la consulta de productos y realización de compras; y **Modo Administrador**, destinado a la gestión y control de las operaciones del emprendimiento.

#### 3.1 Contexto del Emprendimiento y Planteamiento del Problema

Chocolates SV es un emprendimiento salvadoreño dedicado a la elaboración y comercialización de chocolates artesanales personalizados para todo tipo de eventos y ocasiones especiales. Su modelo operativo ha dependido de redes sociales y aplicaciones de mensajería instantánea para la toma de pedidos, cotizaciones y atención al cliente.

Actualmente, el emprendimiento no dispone de una plataforma digital propia. Como resultado esta modalidad genera limitaciones operativas las cuales son:

- Lenta gestión de pedidos.
- Poca promoción de productos.
- Lentitud en la atención en picos de demanda.
- Riesgo de inconsistencia en la toma de pedidos.
- Falta de control en inventario.
- Dificultad para el seguimiento de entregas en fechas clave.

Lo que limita la eficiencia operativa y el alcance comercial del negocio además aumentando los tiempos de respuesta de cara al cliente.

### 3.2 Propósito y Alcance del Sistema

El proyecto busca diseñar y construir un ecosistema digital compuesto por dos aplicaciones frontend independientes en React (Tienda E-Commerce y Panel Backoffice Administrativo) conectadas a un backend centralizado en ASP.NET Core Web API y una base de datos relacional.

- **Alcance del Cliente:** Consulta de catálogo, gestión de promociones/cupones, carrito, checkout con pasarela simulada, confirmación por correo y rastreo público de órdenes mediante ID y correo.
- **Alcance del Administrador:** Control de acceso autenticado, mantenimiento CRUD de catálogo, gestión de pedidos y estados, motor de promociones, dashboard analítico, control de stock mínimo con alertas y agenda de entregas.

### 3.3 Objetivos (General y Específicos)

**Objetivo General:** Desarrollar e implementar una plataforma web de comercio electrónico y gestión administrativa para Chocolates SV mediante una arquitectura basada en microservicios desacoplados (React + ASP.NET Core), que permita automatizar el ciclo de venta, cobro simulado, control de inventario y despacho de pedidos.

#### **Objetivos Específicos:**

1. Diseñar interfaces de usuario intuitivas y responsivas tanto para la tienda pública como para el backoffice administrativo mediante Figma.
2. Implementar una Web API modular y segura con ASP.NET Core y Entity Framework Core que atienda las peticiones de ambos frontends aplicando principios SOLID y Clean Architecture.
3. Construir un motor de promociones y cupones dinámicos que calcule automáticamente los totales en el carrito de compra.
4. Incorporar un flujo de checkout con validación de pagos simulados y notificaciones por correo electrónico.

## 4. Gestión y Metodología de Trabajo

### 4.0.1 Links a los apartados del trabajo (Trello, GitHub, Figma)

Link de Trello: <https://trello.com/b/lh6o8DjT/chocolates-sv-desarrollo>

Link de GitHub (repositorio del proyecto):

<https://github.com/arieldiazdev10/ChocolatesSV.API>

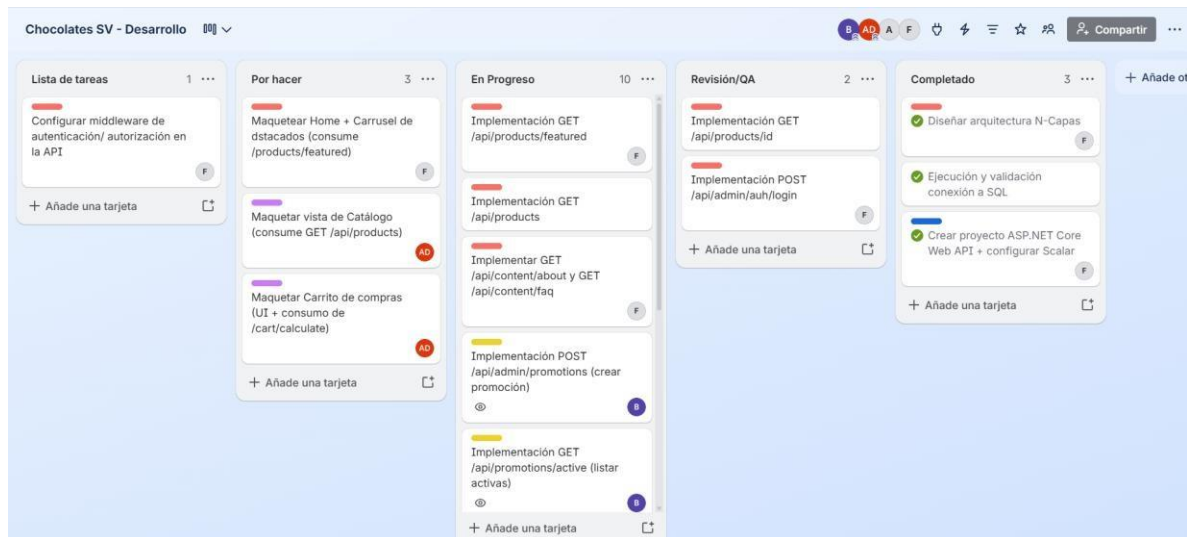
Link de Figma (Diseños de las vistas del proyecto):

<https://www.figma.com/design/7HMEsxegSfLwGAZFOLwCfR/Chocolates-SV?node-id=0-1&t=uNJ3wniNwET6e9Cy-1>

### 4.1 Metodología Ágil (Scrum)

El equipo de desarrollo adopta el marco de trabajo **Scrum** como metodología de gestión ágil, dada su capacidad para estructurar el trabajo en ciclos cortos e iterativos, facilitar la entrega incremental de valor y adaptarse a los cambios en los requisitos del emprendimiento durante el desarrollo del proyecto. Esta elección responde también al requisito de la cátedra de trabajar bajo un marco Scrum o Kanban con roles claramente definidos.

El proyecto se organiza en **dos Sprints**, cada uno alineado con las fechas de entrega establecidas en el cronograma del curso (Fase 1 y Fase 2 respectivamente):



## **Sprint 1 — Fase 1 (Fundamentos y Prototipo Funcional al 50%)**

- Objetivo del sprint: sentar las bases técnicas del sistema y entregar un prototipo funcional que cubra al menos la mitad de las pantallas planificadas, cumpliendo con el porcentaje mínimo exigido para la primera entrega.
- Alcance de trabajo:
- Definición y documentación de los requisitos funcionales y no funcionales del sistema, en base al perfil del proyecto y al feedback recibido del docente.
- Diseño de la arquitectura del backend bajo el patrón N-Capas (Presentación, Aplicación/API, Negocio, Acceso a Datos, Entidades), siguiendo el modelo de referencia de la Guía 4 del curso.
- Modelado y creación del esquema de base de datos relacional en SQL Server, incluyendo tablas, llaves foráneas y datos de prueba iniciales.
- Prototipado de interfaces en Figma para la tienda pública y el panel administrativo (Mockups).
- Desarrollo del primer 50% funcional del sistema, priorizando: ☐
  - ☐ Catálogo de productos (listado, detalle, destacados).
  - ☐ Carrito de compras (cálculo de subtotales).
  - ☐ Autenticación del panel administrativo (login con “Identity”).
  - ☐ CRUD básico de productos y de promociones.

## **Sprint 2 — Fase 2 (Funcionalidad Completa e Integraciones)**

- Objetivo del sprint: completar el alcance funcional restante e integrar los módulos que aportan mayor complejidad al sistema, en respuesta a las funcionalidades adicionales solicitadas por el docente tras la revisión del perfil de proyecto.
- Alcance de trabajo:
- Integración de la pasarela de pago simulada y flujo completo de checkout.

- Envío de notificaciones automáticas por correo electrónico al confirmar un pedido.
- Módulo completo de promociones y cupones, incluyendo validación de códigos y combos.
- Dashboard analítico con indicadores clave (ventas del día/mes, ingresos, pedidos pendientes, top de productos).
- Control de inventario con alertas de stock bajo.
- Seguimiento de pedidos por estado (Pendiente, Confirmado, En Preparación, Enviado, Entregado) y agenda de entregas programadas.
- Pruebas de integración y corrección de errores identificados durante el Sprint 1.
- **Ceremonias Scrum aplicadas:**
- **Sprint Planning:** al inicio de cada sprint, el equipo revisa el backlog priorizado por el Product Owner y define las tareas a desarrollar, distribuyéndolas según el módulo de API asignado a cada desarrollador.
- **Daily Stand-up (asincrónico vía Trello):** dado que el equipo trabaja de forma remota y con horarios de clase, el avance diario se documenta como comentarios o actualizaciones de estado en las tarjetas del tablero, en lugar de una reunión sincrónica diaria.
- **Sprint Review:** al cierre de cada sprint, se presenta el incremento funcional obtenido (el prototipo o la aplicación completa) tanto al Product Owner interno como al docente, en la fecha de defensa correspondiente.
- **Sprint Retrospective:** el equipo se reúne al finalizar cada sprint para identificar qué funcionó bien, qué obstáculos surgieron (por ejemplo, dependencias entre módulos de checkout y promociones) y qué ajustes aplicar en el siguiente ciclo.
- **Definición de "Terminado" (Definition of Done):** una funcionalidad se considera completa cuando su endpoint correspondiente está implementado siguiendo la arquitectura N-Capas, ha sido probado exitosamente en Scalar, está integrado con el frontend correspondiente, y el código ha sido subido a la rama individual del desarrollador en GitHub.

## 4.2 Asignación de Roles del Equipo

| <b>Rol</b>                                        | <b>Integrante</b>                  | <b>Responsabilidad</b>                                                                                                                                                                                                                                                                            |
|---------------------------------------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Product Owner & Frontend Lead (Tienda Cliente)    | Ariel Ernesto Díaz Carpio          | Gestión y priorización del backlog del producto; representa la voz del cliente (Chocolates SV) frente al equipo; desarrollo de la interfaz en React para la tienda pública; consumo de las APIs de catálogo, promociones y carrito de compras.                                                    |
| Scrum Master & Backend Lead                       | Felipe Alberto Salguero Quezada    | Facilitación de las ceremonias Scrum y remoción de obstáculos del equipo; diseño de la arquitectura de la Web API en ASP.NET Core bajo el patrón N-Capas; coordinación técnica de la capa de acceso a datos; implementación de la seguridad basada en “Identity” para el panel administrativo.    |
| Fullstack Developer & Frontend Lead (Panel Admin) | Melany Fabiola Contreras Bojorquez | Desarrollo de la interfaz en React para el Backoffice/CMS administrativo; integración de los CRUDs administrativos (productos, promociones, pedidos) con la API; diseño de interfaz de usuario en Figma para el panel; soporte en la lógica de negocio de los módulos de promociones y dashboard. |

|                        |                               |                                                                                                                                                                                                                                                                                |
|------------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| QA & Database Engineer | Arnulfo Antonio Escobar Valle | Modelado relacional de la base de datos en SQL Server; diseño y ejecución de pruebas unitarias y de integración sobre los endpoints de la API; validación de reglas de negocio; control de versiones y revisión de calidad del código antes de integrarse a la rama principal. |
|------------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 4.3 Gestión de Tareas y Versionamiento

- **Tablero Kanban/Scrum (Trello)**

El equipo gestiona la totalidad de las actividades del proyecto mediante un tablero en Trello, organizado en las siguientes columnas:

Backlog: historias de usuario y tareas técnicas pendientes de iniciar, priorizadas por el Product Owner.

En Progreso: tareas actualmente en desarrollo por algún integrante del equipo.

Revisión/QA: tareas finalizadas por el desarrollador y pendientes de validación funcional por parte del QA Engineer.

Completado: tareas validadas y ya integradas a la rama principal del repositorio.

El tablero se mantiene actualizado durante todo el ciclo del proyecto como evidencia del trabajo realizado, y su enlace público se comparte en el archivo README del repositorio de GitHub, junto con el docente agregado como usuario del tablero, conforme a lo solicitado en los lineamientos de la cátedra.

- **Estrategia de Ramas en Git/GitHub**

Para garantizar la trazabilidad del trabajo individual y facilitar la integración continua del código, se establece el siguiente flujo de ramas:

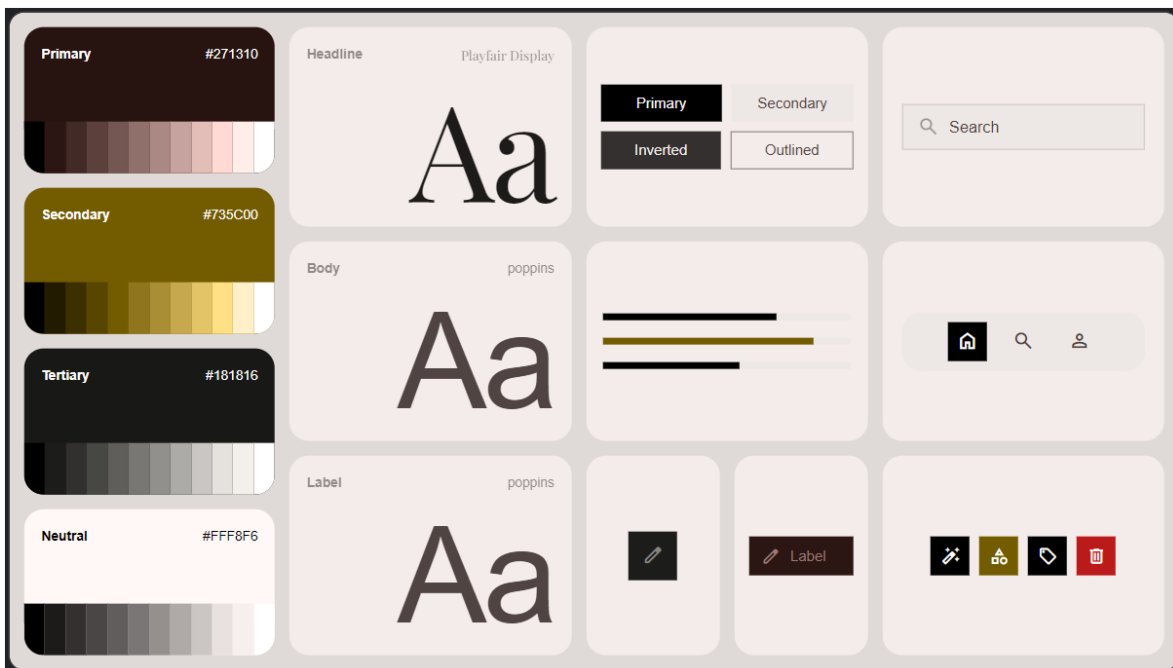
main: rama de producción, contiene únicamente el código estable correspondiente a cada entrega oficial (Fase 1 y Fase 2).

develop: rama de integración, donde convergen los aportes de todos los integrantes antes de fusionarse a main.

Ramas individuales de trabajo (feature/<nombre>-<módulo>): cada integrante del equipo trabaja en su propia rama, evidenciando su contribución individual mediante el historial de commits. Las ramas asignadas son: feature/ariel-storefront — Interfaz de tienda pública. feature/felipe-backend-api — Arquitectura y endpoints del backend. feature/melany-admin-panel — Interfaz y CRUDs del panel administrativo. feature/arnulfo-db-tests — Modelado de base de datos y pruebas. Todos los integrantes están agregados como colaboradores del repositorio, y el archivo README documenta la descripción del proyecto, el enlace al tablero de gestión, los enlaces a los diseños de Figma y el tipo de licencia Creative Commons implementada, conforme a los requisitos de publicación de la cátedra.

## 5. Diseño UX/UI y Prototipado

### 5.1 Guía de Estilo Visual



- **Paleta de Colores:**

- Primario: Marrón Chocolate Profundo (#3E2723) — Evoca el rubro artesanal.
- Secundario/Acento: Dorado Cálido (#D4AF37) — Asociado a promociones, botones de acción primaria y calidad premium.
- Fondo Neutro: Crema Suave (#FDFBF7) — Para fondos limpios que resaltan las fotos de producto.
- Estados: Verde Éxito (#2E7D32), Rojo Alerta/Error (#C62828).
- **Tipografía:** *Poppins* o *Inter* para textos y navegación; *Playfair Display* para encabezados elegantes.
- **Iconografía:** *Lucide React* / *Heroicons* consistentes en navegación, acciones de carrito y estados de orden.

## 5.2 Mockups Principales (Figma)

- **Tienda Pública:** Home con Carrusel de Productos, Vista de Catálogo con Filtros, Vista Detalle de Producto, Carrito de Compras lateral/página, Flujo de Checkout con Pasarela Simulada, Vista de Rastreo de Pedidos.



Artisanal Indulgence

[Inicio](#)[Catálogo](#)[Nuestra Historia](#)[Contacto](#)

Filtros

COLECCIÓN

☒ Trufas Artesanales

☐ Barras Origen Único

☐ Ediciones Limitadas

% CACAO

50%+

70%+

85%+

RANGO DE PRECIO

1%

\$50+

Mostrando 12 resultados

Relevancia

Trufas Noche Mágica

una exquisita colección de trufas de chocolate oscuro 70%...

\$24.00

Origen Único Madagascar

Notas vibrantes de fresa roja y cítricos añaden este sabor de 85%...

\$18.50

EDICIÓN LIMITADA Colección Herencia

Un viaje sensorial a través de nuestra receta más antigua: 24 piezas de indulgencia para elaboradas a mano.

\$85.00

Blanco Pistacho & Frambuesa

Crema de chocolate blanco suave con jugosas gotitas de fresa y...

\$16.00

<12>

Artisanal Indulgence

[Inicio](#)[Catálogo](#)[Nuestra Historia](#)[Contacto](#)

ESTADO DEL ENVÍO

Seguimiento de Pedido

Pedido # AI-847298 • Realizado el 12 de Octubre, 2024

Progreso del Envío

Pedido Confirmado

12 Oct, 10:45 AM

Tu pedido ha sido procesado exitosamente.

En Preparación

13 Oct, 14:20 PM

Nuestros artesanos están empaquetando tus chocolates con cuidado.

En Camino

Hoy, 08:55 AM

El paquete ha salido de nuestra bodega central (Destino estimado: Mariana).

Entregado

Pendiente.

Dirección de Envío

Maria Antonieta

Av. Paseo de la Reforma 250

Apt. 4B, Col. Juárez

Ciudad de México, CDMX 06600

¿Necesitas Ayuda?

Si tienes alguna duda sobre tu envío, nuestro concierge está disponible.

Contactor Concierge

Resumen del Pedido

Colección Noir (70% Cacao)

Cantidad: 1

\$1,250.00

Trufas de Avelana

Cantidad: 2

\$880.00

Subtotal

\$2,140.00

Envío Premium

\$150.00

Total

\$2,290.00

Artisanal Indulgence

Creando momentos de puro placer a través del arte del chocolate artesanal. Tradición y vanguardia en cada bocado.

© 2024 Artisanal Indulgence. El arte del chocolate premium.

LEGAL

Privacidad

Términos de Servicio

ATENCIÓN

Envíos

FAQ

Artisanal Indulgence

El arte del chocolate premium.

© 2024 Artisanal Indulgence. El arte del chocolate premium.

Privacidad

Términos de Servicio

Envíos

FAQ

Artisanal Indulgence

[Inicio](#)[Catálogo](#)[Nuestra Historia](#)

Tu Selección

Trufas Noche Mágica

Caja de 12 unidades • Cacao 70%

\$45.00

- 2 +

Origen Único Madagascar

Tableta 80g • Notas afrutadas

\$18.00

- 1 +

Recomendación del Chocolatier

"Acompaña tus trufas Noche Mágica con un vaso frío robusto o un espresso de especialidad para realzar sus notas oscuras."

Subtotal

\$108.00

Envío Estimado

\$5.00

Total

\$113.00

PROCEDER AL CHECKOUT

Pago seguro garantizado



1. Información de Envío

Ingrese los detalles para la entrega de su pedido.

CONTACTO

Correo electrónico

☐ Deseo recibir noticias y ofertas exclusivas.

DIRECCIÓN DE ENTREGA

Nombre

Apellidos

Dirección (Calle, número)

Apartamento, suite, etc. (opcional)

Código Postal

Ciudad

Provincia/Estado

Teléfono

CONTINUAR AL PAGO

2. Método de Pago

Todas las transacciones son seguras y están encriptadas.



Tarjeta de Crédito

Número de Tarjeta

0000 0000 0000 0000

Nombre en la Tarjeta

Nombre como aparece en la tarjeta

Fecha de Expiración (MM/AA)

MM / AA

Código de Seguridad (CVV)

\*\*\*

[← Volver al Envío](#)

REVISAR PEDIDO

Resumen del Pedido

- Caja Colección  
Herencia  
Praliné de Avellana  
Oscura  
€45.00
- Trufas de Cacao 70%  
Edición Limitada  
€32.00

Código de descuento

Subtotal

€109.00

EnvíoCalculado en el siguiente  
paso

Impuestos

€22.89

Total

EUR **€131.89**

- **Panel Administrador:** Pantalla de Login, Dashboard de Métricas, Tabla de Gestión de Productos (CRUD), Módulo de Pedidos y Agenda de Despachos.



**Chocolates SV**  
Artisanal Excellence

+ Create New Order

- Dashboard
- Inventory**
- Orders
- Customers
- Promotions

- Settings
- Logout

## Inventory Management

Manage product catalog, track stock levels, and monitor status.

Search Products

Category

Stock Status

Search by name, SKU...

All Categories

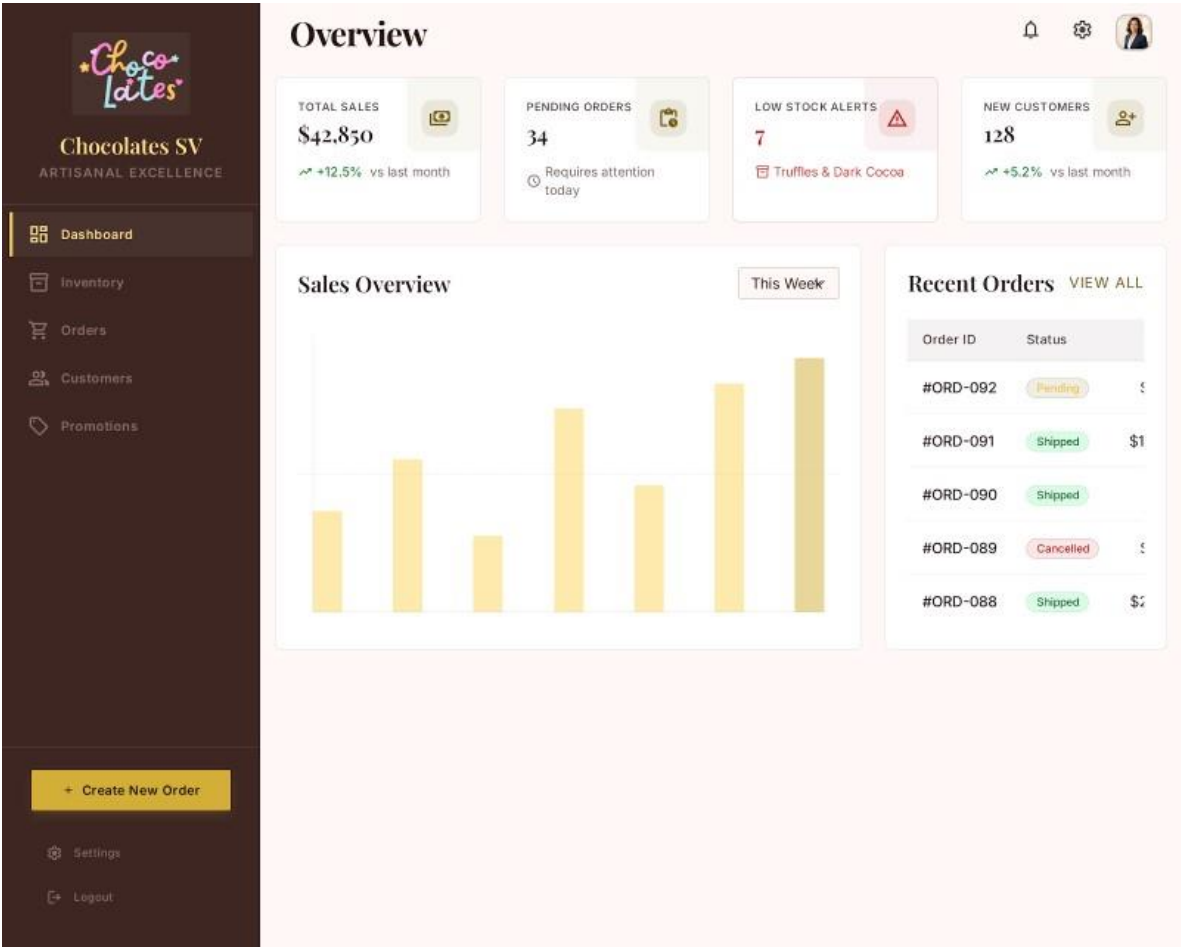
All Statuses

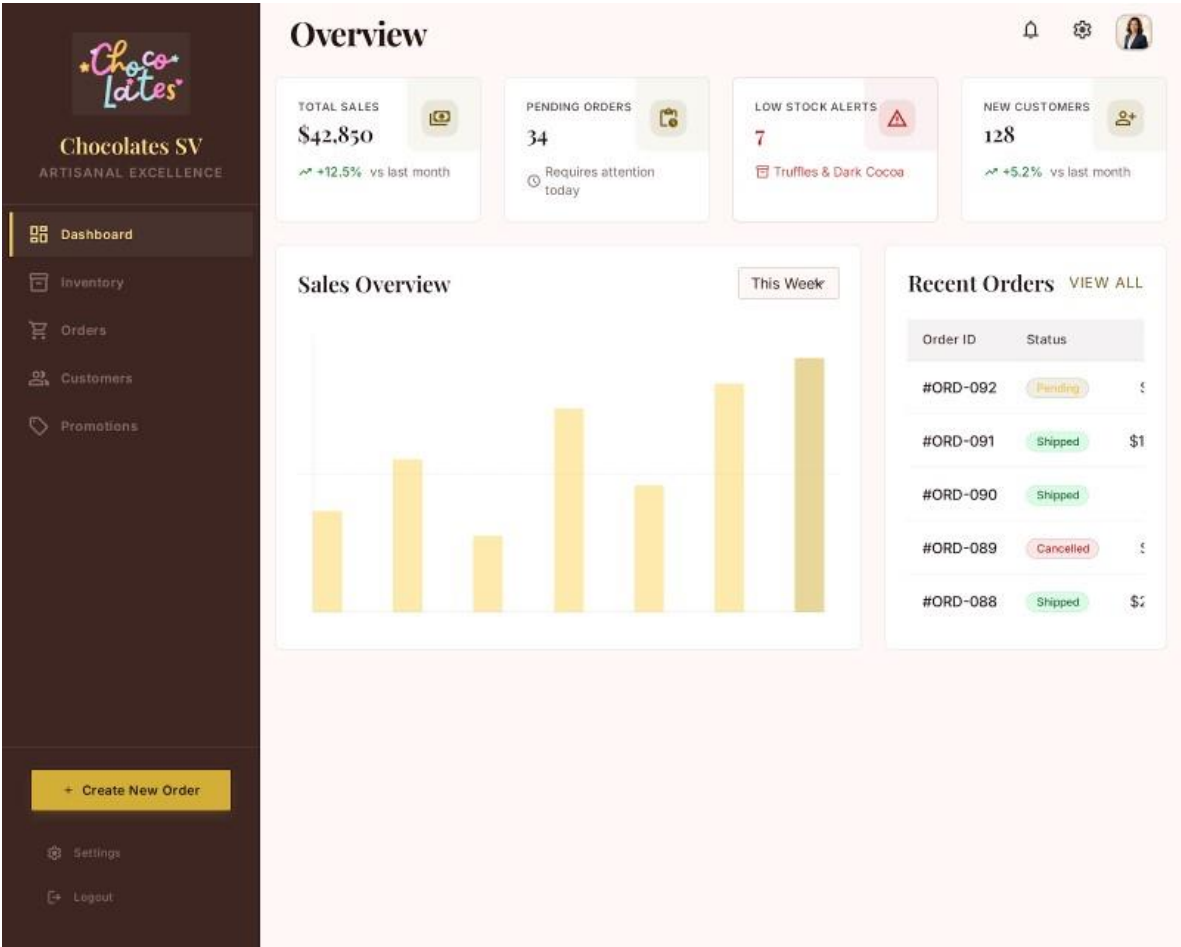
| Product Name                                                                                                                        | Category       | Price   | Stock Level | Status    | Actions |
|-------------------------------------------------------------------------------------------------------------------------------------|----------------|---------|-------------|-----------|---------|
|  <b>70% Dark Chocolate Bar</b><br>SKU: CB-DRK-01   | Chocolate Bars | \$8.50  | 145 units   | In Stock  |         |
|  <b>Sea Salt Caramel Truffle</b><br>SKU: TR-SSC-01 | Truffles       | \$3.25  | 12 units    | Low Stock |         |
|  <b>Artisanal Selection Box</b><br>SKU: GB-ART-12 | Gift Boxes     | \$45.00 | 34 units    | In Stock  |         |

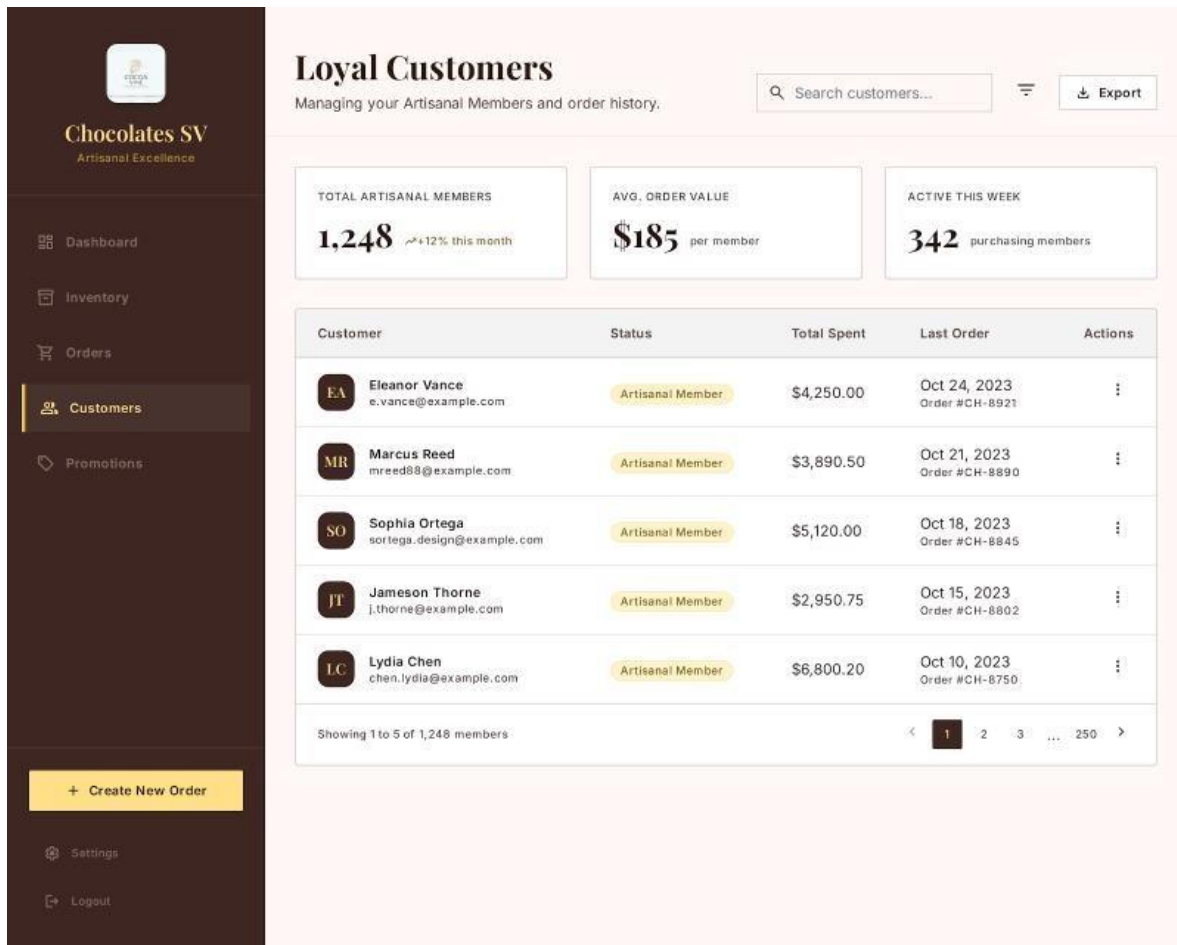
Showing 1-10 of 42 products

<

>







## 6. Estrategia de Solución y Arquitectura Técnica

### 6.1. Arquitectura General del Sistema

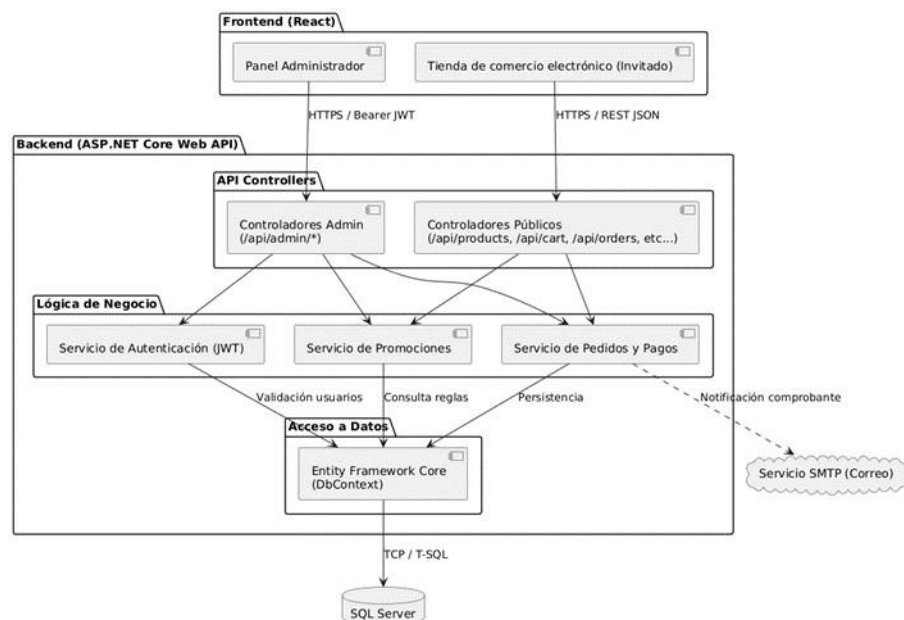
El ecosistema digital de Chocolates SV se fundamenta en una Arquitectura Cliente-Servidor desacoplada basada en APIs RESTful. Se diseñaron dos aplicaciones independientes de una sola página (*Single Page Applications - SPA*) desarrolladas en React, las cuales consumen un backend unificado desarrollado en ASP.NET Core Web API (.NET 10) conectado a una base de datos relacional en Microsoft SQL Server:

- 1. Frontend Página de Comercio Electrónico (Modo Cliente / Invitado):**  
Aplicación pública orientada al consumidor final. Opera bajo un modelo sin autenticación previa, interactuando con los endpoints públicos de la API

(/api/products/\*, /api/promotions/\*, /api/cart/\*, /api/checkout/\*, /api/orders/\*, /api/content/\*) mediante peticiones asíncronas vía HTTPS.

2. **Frontend Panel de Administración (Modo Administrador):** Aplicación de uso interno para el personal del emprendimiento. Requiere autenticación basada en “Identity” y consume exclusivamente la familia de endpoints administrativos (/api/admin/\*).
3. **Backend Centralizado (ASP.NET Core Web API):** Orquesta las reglas de negocio, procesa la simulación de pagos, calcula los descuentos por cupón/oferta, ejecuta la persistencia mediante Entity Framework Core y despacha notificaciones automáticas por correo electrónico.

### Diagrama de componentes de plataforma de comercio electrónico y panel de administración para Chocolates SV



## 6.2 Patrón Arquitectónico del Backend: Arquitectura en N-Capas

Para el desarrollo del backend centralizado en ASP.NET Core Web API, se adoptó el modelo de Arquitectura en N-Capas. Este modelo organiza el software en múltiples proyectos de biblioteca de clases (*Class Libraries*) con responsabilidades delimitadas y bajo acoplamiento, comunicándose secuencialmente mediante interfaces y contratos formales.

### 1. ChocolatesSV.API (Capa de Presentación / UI Web API)

- **Responsabilidad:** Exponer los endpoints RESTful para ambos frontends (Tienda de comercio electrónico y Panel administrativo), recibir las solicitudes HTTP, aplicar validaciones de entrada, gestionar los códigos de estado HTTP (200 OK, 201 Created, 400 BadRequest, 404 NotFound) y registrar los servicios en el contenedor de dependencias (Program.cs).
- **Especificación y Exploración con Scalar:** Se implementó la suite **Scalar** (Scalar.AspNetCore y Microsoft.AspNetCore.OpenApi) como reemplazo estándar de Swagger en .NET para generar la especificación OpenAPI y proporcionar una interfaz interactiva de pruebas en tiempo de desarrollo (/scalar/).

### 2. ChocolatesSV.BL (Capa de Lógica de Negocio / Business Logic)

- **Responsabilidad:** Implementar la lógica empresarial, cálculo de totales, validación de reglas de promociones y cupones, y orquestar el flujo entre los controladores y la capa de datos.
- **Uso de AutoMapper:** Para evitar la exposición directa de la estructura interna de la base de datos y proteger la integridad del dominio, se implementa **AutoMapper** mediante perfiles (Profile), automatizando la transformación bidireccional entre los modelos de base de datos (Models) y los objetos de transferencia de datos (DTOs).

### 3. ChocolatesSV.DAL (Capa de Acceso a Datos / Data Access Layer)

- **Responsabilidad:** Gestionar la comunicación con Microsoft SQL Server mediante conexiones seguras y transacciones.

- **Uso de Dapper (Micro-ORM):** Se seleccionó **Dapper** junto con `Microsoft.Data.SqlClient` por su alto rendimiento, mínima sobrecarga y capacidad de ejecutar sentencias SQL nativas parametrizadas (`QueryAsync<T>`, `QueryFirstOrDefaultAsync<T>`, `ExecuteAsync`, `ExecuteScalarAsync`), previniendo vulnerabilidades de inyección SQL.
- **Patrón Repositorio Base (IDatabaseRepository):** Centraliza la apertura de conexiones asíncronas, el registro de errores en consola (*Logging*) y la gestión de transacciones atómicas (`BeginTransactionAsync`).

#### 4. ChocolatesSV.Entities (Entidades de Negocio y Contratos)

- **Carpeta Models:** Clases POCO con decoradores de clave primaria (`[Key]`) que representan las tablas físicas del esquema relacional (Productos, Categorías, Pedidos, DetallesPedido, Promociones, Usuarios).
- **Carpeta DTO (Data Transfer Objects):** Clases optimizadas para la transferencia de datos hacia los frontends, incorporando reglas de validación semántica con *DataAnnotations* (`[Required]`, `[StringLength]`, `[Range]`).

#### 5. ChocolatesSV.Common (Componentes Transversales)

- **Responsabilidad:** Contener clases de configuración compartida entre todas las capas, destacando `AppSettings`, la cual enlaza la cadena de conexión (`ConnectionString`) tipada desde `appsettings.json` mediante el patrón `IOptions<AppSettings>`.

##### 6.2.2 Inyección de Dependencias y Modularidad (Extension Methods)

Para mantener una arquitectura limpia y respetar el principio de Inversión de Dependencias (DIP), cada capa gestiona su propio registro de servicios mediante clases estáticas de extensión (`ServiceCollectionExtensions`):

- **En DAL (AddRepositoryConnector):** Registra con ciclo de vida transitorio (`AddTransient`) el repositorio base `IDatabaseRepository` y los repositorios específicos (`IProductoRepository`, `IPedidoRepository`, `IPromocionRepository`, etc.).

- **En BL (AddServiceConnector):** Registra los perfiles de AutoMapper (AddAutoMapper) y los servicios de dominio (IProductoService, IPedidoService, etc.).
- **En API (Program.cs):** Consume ambos conectores de forma concisa antes de la construcción de la aplicación web (builder.Build()).

### 6.3 Mapeo y Flujo de las APIs RESTful

El backend organiza sus servicios en dos grupos funcionales:

#### 6.3.1 APIs del Modo Cliente (Sin Cuenta / Invitado)

| Endpoint                       | Verbo HTTP | Descripción y Regla de negocio                                                |
|--------------------------------|------------|-------------------------------------------------------------------------------|
| /api/products/featured         | GET        | Recupera chocolates marcados con flag destacado para el carrusel inicial.     |
| /api/products                  | GET        | Lista productos activos con soporte para filtrado por categoría y paginación. |
| /api/products/{id}             | GET        | Retorna el detalle completo de un chocolate específico.                       |
| /api/promotions/active         | GET        | Obtiene promociones de temporada y combos vigentes.                           |
| /api/promotions/validatecoupon | POST       | Valida código de cupón contra fecha de expiración y estado de activación.     |
| /api/cart/calculate            | POST       | Recibe el arreglo de ítems del carrito y calcula subtotales,                  |

|                               |      |                                                                                                                                                  |
|-------------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------|
|                               |      | descuentos y total neto.                                                                                                                         |
| /api/checkout/simulatepayment | POST | Valida la estructura de la tarjeta ficticia y genera una respuesta simulada con token de autorización                                            |
| /api/orders                   | POST | Registra el pedido en base de datos, reduce existencias de inventario, genera el código correlativo de orden y dispara el correo de confirmación |
| /api/orders/tracking          | POST | Valida número de pedido y correo electrónico para exponer el estado actual del despacho sin requerir login.                                      |
| /api/content/about            | GET  | Provee la información institucional de Chocolates SV.                                                                                            |
| /api/content/faq              | GET  | Entrega el listado parametrizado de preguntas frecuentes.                                                                                        |

### 6.3.2 APIs del Modo Administrador (Autenticación con “Identity”)

| Endpoint | Verbo HTTP | Descripción y Regla de negocio |
|----------|------------|--------------------------------|
|----------|------------|--------------------------------|

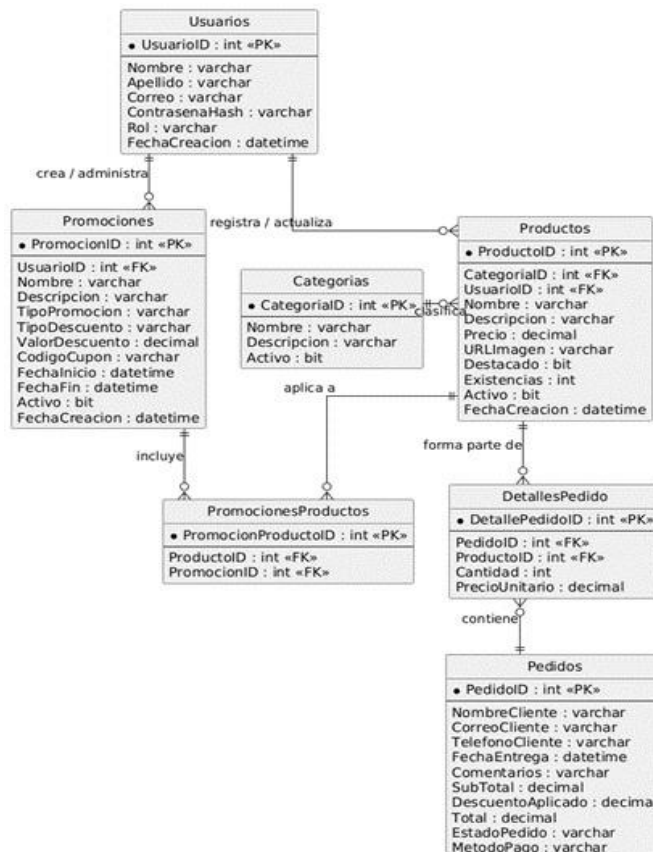
|                            |        |                                                                                                                                  |
|----------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------|
| /api/admin/auth/login      | POST   | utiliza un servicio interno llamado "SignInManager" que valida las credenciales del usuario contra una base de datos (Identity). |
| /api/admin/products        | POST   | Registra un nuevo chocolate en el catálogo                                                                                       |
| /api/admin/products/{id}   | PUT    | Actualiza la información técnica,                                                                                                |
|                            |        | precios y existencias de un chocolate.                                                                                           |
| /api/admin/products/{id}   | DELETE | Ejecuta el borrado lógico/desactivación del producto.                                                                            |
| /api/admin/promotions      | POST   | Configura una nueva oferta, combo o cupón con rango de fechas.                                                                   |
| /api/admin/promotions/{id} | PUT    | Modifica parámetros o suspende una promoción existente                                                                           |
| /api/admin/promotions/{id} | DELETE | Elimina la promoción del registro.                                                                                               |
| /api/admin/orders          | GET    | Lista todos los pedidos con soporte de filtros por rango de fechas y estado de entrega                                           |
| /api/admin/orders/{id}     | GET    | Muestra el desglose detallado de la orden, datos de entrega y comprobante.                                                       |

|                                  |       |                                                                                            |
|----------------------------------|-------|--------------------------------------------------------------------------------------------|
| /api/admin/orders/{id}/status    | PATCH | Actualiza el estado de la orden en el flujo logístico.                                     |
| /api/admin/dashboard/summary     | GET   | Consolida métricas como: ventas diarias, acumulado mensual, ingresos y pedidos pendientes. |
| /api/admin/dashboard/topproducts | GET   | Ejecuta agregaciones SQL para listar los chocolates de mayor rotación.                     |

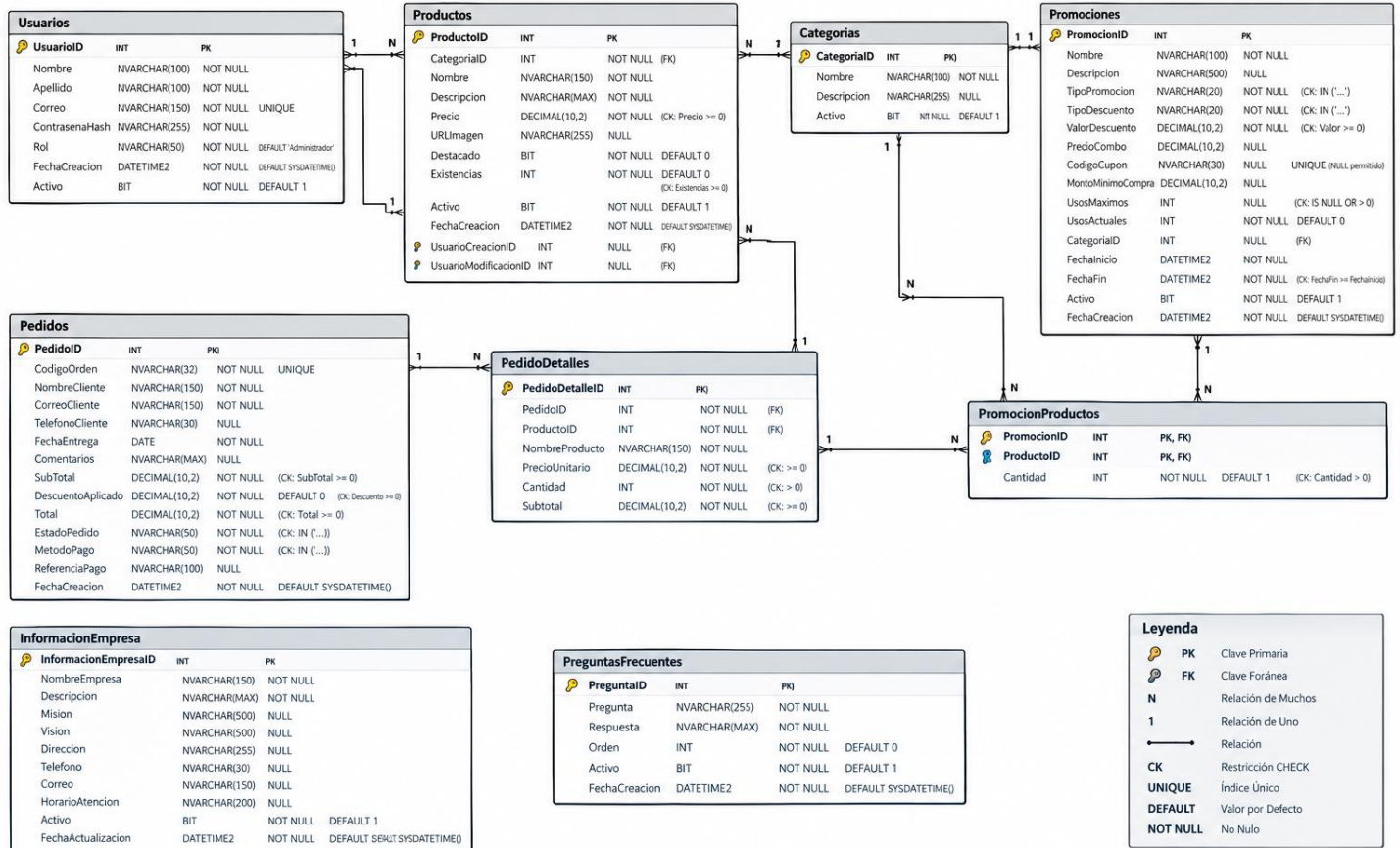
## 6.4. Modelado de Datos

La persistencia de datos del sistema Chocolates SV se implementa en Microsoft SQL Server mediante un esquema relacional normalizado que soporta el ciclo transaccional de compras, inventario, categorización y promociones.

### 6.4.1 Diagrama Entidad-Relación



## 6.4.2 Diagrama de base de datos



## 6.4.3 Diccionario de base de datos

| Tabla     |           | Usuarios                                                                                                               |          |      |       |                   |             |
|-----------|-----------|------------------------------------------------------------------------------------------------------------------------|----------|------|-------|-------------------|-------------|
| Propósito |           | Almacena exclusivamente las credenciales y perfiles de los administradores a la plataforma del panel de administración |          |      |       |                   |             |
| Campo     | Tipo dato | de                                                                                                                     | Longitud | Nulo | Llave | Valor por defecto | Descripción |

|                |          |         |    |    |                 |                                                    |
|----------------|----------|---------|----|----|-----------------|----------------------------------------------------|
| UsuarioID      | INT      | 4 bytes | NO | PK | IDENTITY(1,1)   | Identificador único autoincremental del usuario    |
| Nombre         | NVARCHAR | 100     | NO | -  | -               | Nombre(s) del usuario administrador                |
| Apellido       | NVARCHAR | 100     | NO | -  | -               | Apellido(s) del usuario administrador              |
| Correo         | NVARCHAR | 150     | NO | UQ | -               | Correo electrónico de acceso (único en el sistema) |
| ContraseñaHash | NVARCHAR | 255     | NO | -  | -               | Hash criptográfico de la contraseña                |
| Rol            | NVARCHAR | 50      | SÍ | -  | 'Administrador' | Nivel de permisos dentro del panel de control      |
| FechaCreación  | DATETIME | 8bytes  | SÍ | -  | GETDATE()       | Fecha y hora exacta de registro                    |

|        |     |      |    |   |   |                                                                     |
|--------|-----|------|----|---|---|---------------------------------------------------------------------|
| Activo | BIT | 1bit | SÍ | - | 1 | Estado del usuario dentro de la plataforma (1: Activo, 0: Inactivo) |
|--------|-----|------|----|---|---|---------------------------------------------------------------------|

| Tabla        |              | Categorias                                                                                             |      |       |                   |                                                       |  |
|--------------|--------------|--------------------------------------------------------------------------------------------------------|------|-------|-------------------|-------------------------------------------------------|--|
| Propósito    |              | Catálogo para clasificar y organizar los productos en la tienda en línea y en el panel administrativo. |      |       |                   |                                                       |  |
| Campo        | Tipo de dato | Longitud                                                                                               | Nulo | Llave | Valor por defecto | Descripción                                           |  |
| Categoría ID | INT          | 4 bytes                                                                                                | NO   | PK    | IDENTITY(1,1)     | Identificador único auto incremental de la categoría  |  |
| Nombre       | NVARCHAR     | 100                                                                                                    | NO   | -     | -                 | Nombre descriptivo de la categoría                    |  |
| Descripción  | NVARCHAR     | 100                                                                                                    | NO   | -     | -                 | Detalle complementario sobre la clasificación.        |  |
| Activo       | BIT          | 1 bit                                                                                                  | SÍ   | -     | 1                 | Borrado lógico (1: Visible en el catálogo, 0: Oculta) |  |

| Tabla          |              | Promociones                                                                                                           |        |       |                   |                                                                             |  |
|----------------|--------------|-----------------------------------------------------------------------------------------------------------------------|--------|-------|-------------------|-----------------------------------------------------------------------------|--|
| Propósito      |              | Almacena la configuración de campañas promocionales, temporadas, combos y cupones de descuento aplicables al carrito. |        |       |                   |                                                                             |  |
| Campo          | Tipo de dato | Longitud                                                                                                              | Número | Llave | Valor por defecto | Descripción                                                                 |  |
| PromocionID    | INT          | 4 bytes                                                                                                               | NO     | PK    | IDENTITY(1,1)     | Identificador único auto incremental de la promoción.                       |  |
| Nombre         | NVARCHAR     | 100                                                                                                                   | NO     | -     | -                 | Nombre comercial de la oferta o campaña.                                    |  |
| Descripcion    | NVARCHAR     | 255                                                                                                                   | SÍ     | -     | -                 | Términos o detalles de la promoción.                                        |  |
| TipoPromocion  | NVARCHAR     | 50                                                                                                                    | NO     | -     | -                 | ('Temporada', 'DescuentoCategoría', 'Combo', 'Cupon') (único en el sistema) |  |
| TipoDescuento  | DECIMAL      | (10,2)                                                                                                                | NO     | -     | -                 | ('Porcentaje', 'MontoFijo')                                                 |  |
| ValorDescuento | NVARCHAR     | 50                                                                                                                    | SÍ     | -     | -                 | Porcentaje (\$0.01-100.00\$) o monto en USD a descontar.                    |  |

|                       |           |         |    |    |           |                                                                      |
|-----------------------|-----------|---------|----|----|-----------|----------------------------------------------------------------------|
| CodigoCupon           | NVARCHAR  | 50      | SÍ | UQ | -         | Código alfanumérico ingresado por el cliente.                        |
| FechaInicio           | DATE TIME | 8 bytes | NO | -  | -         | Fecha y hora en la que inicia la vigencia                            |
| FechaFin              | DATE TIME | 8 bytes | NO | -  | -         | Fecha y hora límite de expiración                                    |
| Activo                | BIT       | 1bit    | SÍ | -  | 1         | Estado de la oferta dentro de la plataforma (1: Activo, 0: Inactivo) |
| FechaCreacion         | DATE TIME | 8 bytes | SÍ | -  | GETDATE() | Fecha de registro en el sistema.                                     |
| UsuarioCreacionID     | INT       | 4 bytes | SÍ | FK | -         | Usuario administrador que registró la oferta (Usuarios.UsuarioID).   |
| UsuarioModificacionID | INT       | 4 bytes | SÍ | FK | -         | Último administrador que editó la oferta (Usuarios.UsuarioID).       |

|              |           |
|--------------|-----------|
| <b>Tabla</b> | Productos |
|--------------|-----------|

| Propósito   |              | Catálogo principal de chocolates con control de existencias, imágenes y auditoría de cambios. |        |       |                   |                                                          |
|-------------|--------------|-----------------------------------------------------------------------------------------------|--------|-------|-------------------|----------------------------------------------------------|
| Campo       | Tipo de dato | Longitud                                                                                      | Número | Llave | Valor por defecto | Descripción                                              |
| ProductoID  | INT          | 4 bytes                                                                                       | NO     | PK    | IDENTITY(1,1)     | Identificador único auto incremental del producto        |
| CategoriaID | INT          | 4 bytes                                                                                       | NO     | FK    | -                 | Categoría a la que pertenece (Categorias.CategoriaID)    |
| Nombre      | NVARCHAR     | 150                                                                                           | NO     | -     | -                 | Nombre comercial del chocolate                           |
| Descripcion | NVARCHAR     | MAX                                                                                           | NO     | -     | -                 | Descripción detallada, notas de sabor e ingredientes.    |
| Precio      | DECIMAL      | (10,2)                                                                                        | NO     | -     | -                 | Precio base de venta al público en USD.                  |
| URLImagen   | NVARCHAR     | 255                                                                                           | SÍ     | -     | -                 | Ruta o enlace de la imagen principal del producto.       |
| Destacado   | BIT          | 1 bit                                                                                         | SÍ     | -     | 0                 | Flag para mostrar en el carrusel de Home (1: Sí, 0: No). |
| Existencias | INT          | 4 bytes                                                                                       | SÍ     | -     | 0                 | Cantidad física disponible en inventario.                |

|                       |          |         |    |    |           |                                                                  |
|-----------------------|----------|---------|----|----|-----------|------------------------------------------------------------------|
| Activo                | BIT      | 1 bit   | SÍ | -  | 1         | Borrado lógico (1:                                               |
|                       |          |         |    |    |           | Activo/Visible, 0: Desactivado).                                 |
| FechaCreacion         | DATETIME | 8 bytes | SÍ | -  | GETDATE() | Fecha de registro en el sistema.                                 |
| UsuarioCreacionID     | INT      | 4 bytes | SÍ | FK | -         | Usuario administrador que creó el producto (Usuarios.UsuarioID). |
| UsuarioModificacionID | INT      | 4 bytes | SÍ | FK | -         | Último administrador que editó el producto (Usuarios.UsuarioID). |

| Tabla                   |  | PromocionesProductos                                                                            |              |          |           |                         |                                                                    |
|-------------------------|--|-------------------------------------------------------------------------------------------------|--------------|----------|-----------|-------------------------|--------------------------------------------------------------------|
| Propósito               |  | Tabla asociativa () que vincula promociones específicas con uno o varios productos del catálogo |              |          |           |                         |                                                                    |
| Campo                   |  | Tip<br>o<br>de<br>dat<br>o                                                                      | Longit<br>ud | Nul<br>o | Llav<br>e | Valor<br>por<br>defecto | Descripción                                                        |
| PromocionProduct<br>oID |  | INT                                                                                             | 4<br>bytes   | NO       | PK        | IDENTITY(1<br>,1)       | Identificador único auto<br>incremental de la<br>relación          |
| ProductoID              |  | INT                                                                                             | 4<br>bytes   | NO       | FK        | -                       | Identificador del<br>producto<br>(Productos.ProductoID).           |
| PromocionID             |  | INT                                                                                             | 4<br>bytes   | NO       | FK        | -                       | Identificador de la<br>promoción<br>(Promociones.Promocio<br>nID). |

| Tabla |  | Pedidos |  |  |  |  |  |
|-------|--|---------|--|--|--|--|--|
|-------|--|---------|--|--|--|--|--|

| Propósito         |              | Registro de cabecera de las órdenes generadas por clientes invitados, importes totales y seguimiento logístico. |      |       |                   |                                                        |
|-------------------|--------------|-----------------------------------------------------------------------------------------------------------------|------|-------|-------------------|--------------------------------------------------------|
| Campo             | Tipo de dato | Longitud                                                                                                        | Nulo | Llave | Valor por defecto | Descripción                                            |
| PedidoID          | INT          | 4 bytes                                                                                                         | NO   | PK    | IDENTITY(1,1)     | Identificador único y correlativo de la orden          |
| NombreCliente     | NVARCHAR     | 150                                                                                                             | NO   | -     | -                 | Nombre completo del comprador                          |
| CorreoCliente     | NVARCHAR     | 150                                                                                                             | NO   | -     | -                 | Correo para envío de comprobante y rastreo público.    |
| TelefonoCliente   | NVARCHAR     | 20                                                                                                              | SÍ   | -     | -                 | Teléfono de contacto para la entrega.                  |
| FechaEntrega      | DATE         | 3 bytes                                                                                                         | NO   | -     | -                 | Fecha programada para entrega/despacho.                |
| Comentarios       | NVARCHAR     | MAX                                                                                                             | SÍ   | -     | -                 | Instrucciones especiales de personalización o entrega. |
| Subtotal          | DECIMAL      | (10,2)                                                                                                          | NO   | -     | -                 | Monto total deducido por promociones/cupones           |
| DescuentoAplicado | DECIMAL      | (10,2)                                                                                                          | SÍ   | -     | -                 | Fecha y hora en la que inicia la vigencia              |
| Total             | DECIMAL      | (10,2)                                                                                                          | NO   | -     | -                 | Monto final a pagar (SubTotal - DescuentoAplicado)     |

|                |          |         |    |   |              |                                                                                    |
|----------------|----------|---------|----|---|--------------|------------------------------------------------------------------------------------|
| EstadoPedido   | NVARCHAR | 50      | SÍ | - | 'Confirmado' | ('Pendiente', 'Confirmado', 'En Preparacion', 'Enviado', 'Entregado', 'Cancelado') |
| MetodoPago     | NVARCHAR | 50      | NO | - | -            | ('Simulado', 'ContraEntrega', 'TransferenciaBanca ria')                            |
| ReferenciaPago | NVARCHAR | 100     | SÍ | - | -            | Código de transacción generado por la pasarela simulada o comprobante.             |
| FechaCreacion  | DATETIME | 8 bytes | SÍ | - | GETDATE()    | Fecha y hora en la que se completó el checkout                                     |

| Tabla           |           | DetallesPedido                                                                                             |      |       |                   |                                                  |  |
|-----------------|-----------|------------------------------------------------------------------------------------------------------------|------|-------|-------------------|--------------------------------------------------|--|
| Propósito       |           | Almacena el desglose de productos, cantidades y precios unitarios históricos correspondientes a cada orden |      |       |                   |                                                  |  |
| Campo           | Tipo dato | Longitud                                                                                                   | Nulo | Llave | Valor por defecto | Descripción                                      |  |
| DetallePedidoID | INT       | 4 bytes                                                                                                    | NO   | PK    | IDENTITY(1,1)     | Identificador único del renglón de pedido.       |  |
| PedidoID        | INT       | 4 bytes                                                                                                    | NO   | FK    | -                 | Pedido padre al que pertenece (Pedidos.PedidoID) |  |
| ProductoID      | INT       | 4 bytes                                                                                                    | NO   | FK    | -                 | Producto adquirido (Productos.ProductoID).       |  |

|                |         |         |    |   |   |                                                  |
|----------------|---------|---------|----|---|---|--------------------------------------------------|
| Cantidad       | INT     | 4 bytes | NO | - | - | Unidades compradas                               |
| PrecioUnitario | DECIMAL | (10,2)  | NO | - | - | Precio unitario congelado al momento de la venta |
| Subtotal       | DECIMAL | (10,2)  | NO | - | - | Importe calculado (Cantidad * PrecioUnitario)    |

#### 6.4.4 Estructura en ChocolatesSV.Entities (Models y DTOs)

Se mantiene una estricta separación entre los modelos de base de datos (Models) y los contratos expuestos al exterior (DTOS).

- **Modelos de Dominio (ChocolatesSV.Entities.Models):** Clases POCO equivalentes a las tablas relacionales, decoradas con [Key] de **System.ComponentModel.DataAnnotations**.
- **DataTransferObjects (ChocolatesSV.Entities.DTO):** Clases que transfieren datos con validaciones declarativas ([Required], [StringLength], [Range], etc.), ocultando campos internos y desacoplando la nomenclatura de la base de datos.

#### 6.4.5 Mapeo de Objetos con AutoMapper(ChocolatesSV.BL.Profiles)

Para transformar de manera eficiente y automática los modelos a DTOs y viceversa sin transferir código manualmente, se implementan perfiles derivados de Profile en la capa de negocio.

#### 6.4.6 Persistencia con Micro-ORM Dapper (ChocolatesSV.DAL)

La persistencia no utiliza sobrecargas pesadas de ORMs tradicionales, sino el enfoque de alto rendimiento de **Dapper**.

- **Repositorio Genérico Base (IDatabaseRepository / DatabaseRepository)**

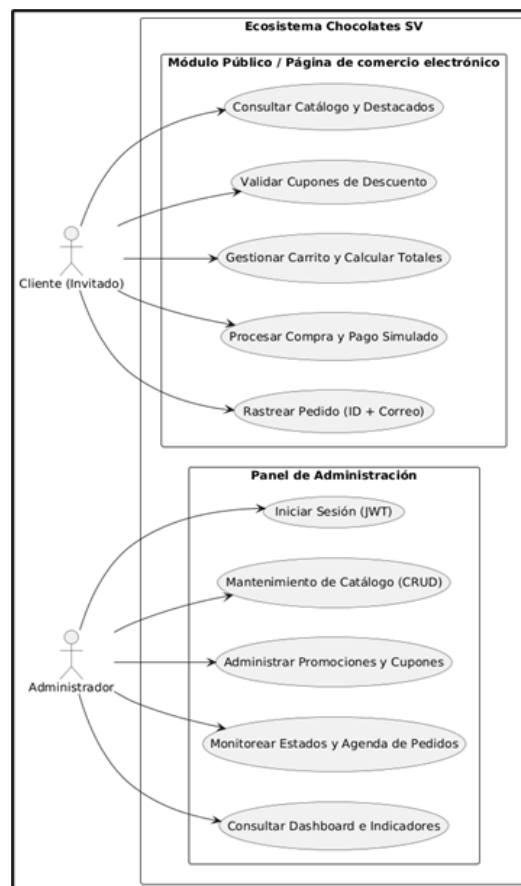
Centraliza el manejo de conexiones SqlConnection, la ejecución segura de sentencias T-SQL parametrizadas y el control de transacciones ACID (BeginTransactionAsync)

- **Implementación de Repositorio Específico (Ejemplo: ProductoRepository)**

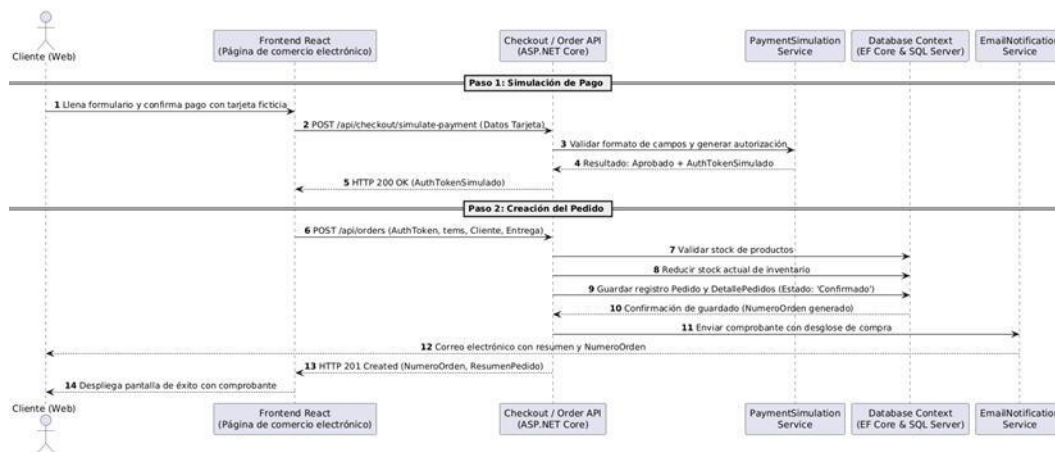
Ejecuta comandos SQL nativos con parámetros fuertemente tipados para mitigar riesgos de SQL Injection.

## 6.5. Diagramas UML de Comportamiento y Estructura

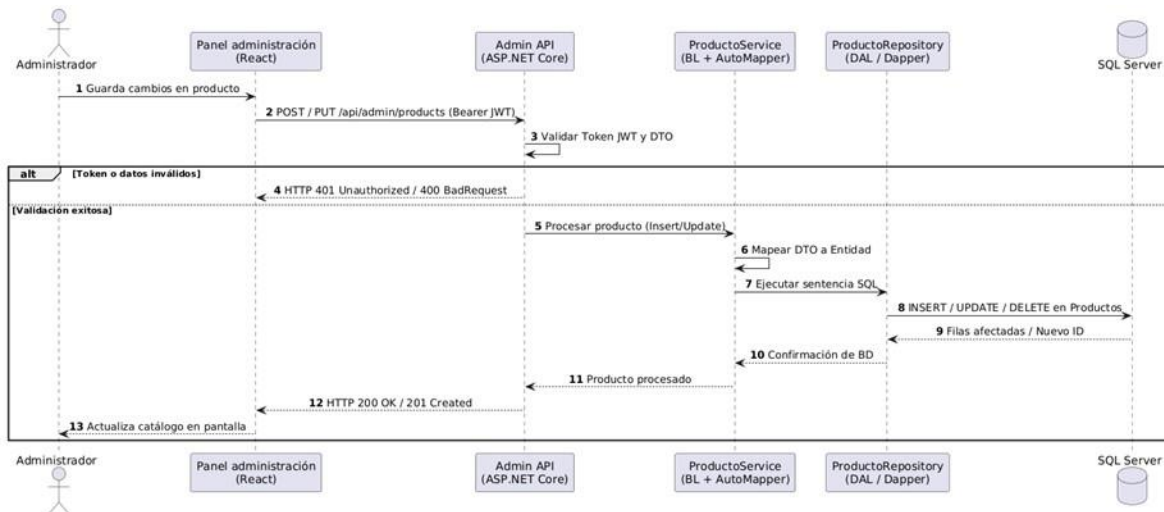
### 6.5.1 Diagrama de Casos de Uso (Cliente vs Administrador)



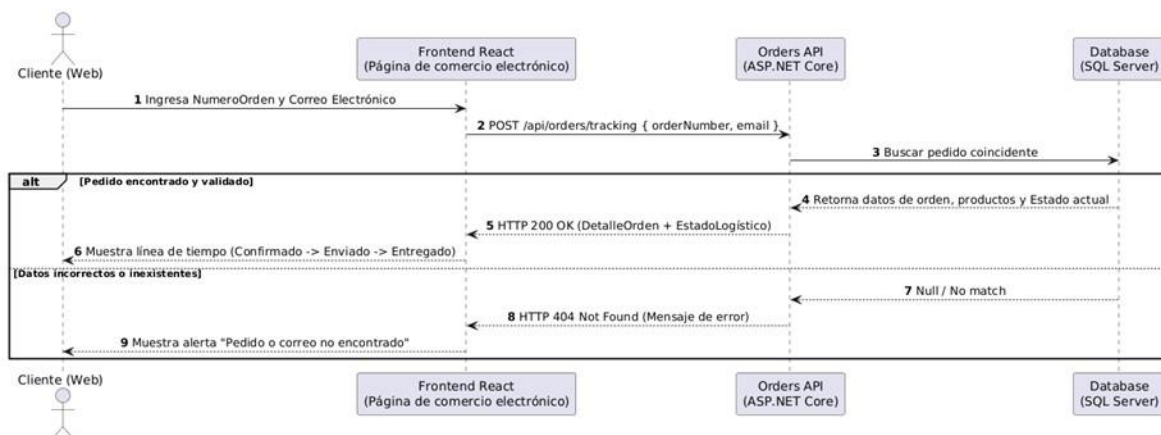
## 6.5.2 Diagramas de Secuencia clave Proceso de Checkout / Compra



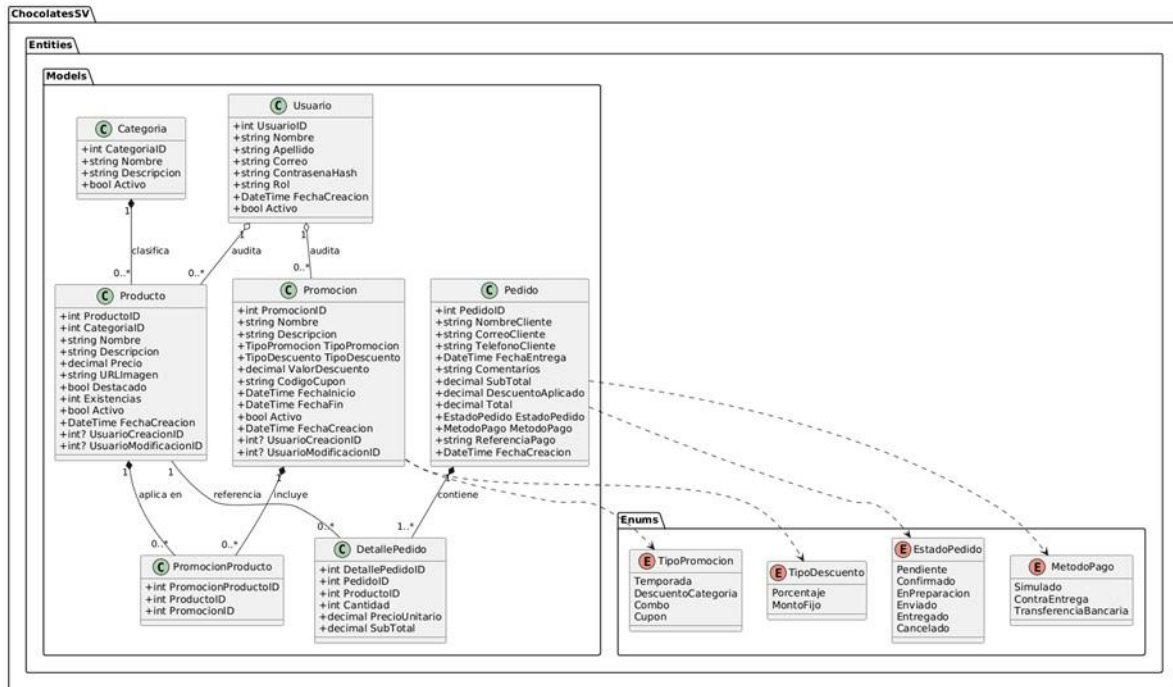
## 6.5.3 Diagrama de secuencia para el mantenimiento del catálogo



## 6.5.4 Diagrama de secuencia para rastreo de pedido de cliente (Invitado)



## 6.5. Diagrama de Clases del Dominio



## 6.6. Estrategia de Seguridad y Validaciones

### 1. Protección de Rutas Administrativas:

- Las rutas `/api/admin/*` implementan el atributo `[Authorize]` en ASP.NET Core.
- La identidad del administrador se transmite en el encabezado `Authorization`:
- Las contraseñas de los usuarios administradores se almacenan cifradas en la base de datos mediante funciones de derivación segura (*BCrypt*).

### 2. CORS (Cross-Origin Resource Sharing):

- Configuración de políticas de CORS restrictivas en ASP.NET Core para permitir únicamente las solicitudes provenientes de los dominios/puertos autorizados de ambos frontends en React (Tienda y Backoffice).

### 3. Validación de Entradas:

- Implementación de *Data Annotations* y filtros de modelo para asegurar que las solicitudes hacia endpoints críticos

(/api/cart/calculate, /api/checkout/simulate-payment, /api/orders, /api/orders/tracking) contengan formatos válidos de correo, campos numéricos positivos y estructuras correctas, previniendo inyecciones de código y datos inconsistentes.

## 7. Stack Tecnológico y Herramientas

- **Frontend (Tienda & Admin — React, Vite, Tailwind CSS):** Se crean dos proyectos independientes (*Single Page Applications*) configurados con **Vite** para optimizar el empaquetado y la velocidad de desarrollo. Las interfaces de usuario de la tienda y del panel administrativo consumen las APIs REST mediante clientes HTTP asíncronos (fetch o axios), mientras que **Tailwind CSS** se emplea para estructurar un diseño visual responsivo y adaptable mediante clases utilitarias.
- **Backend (ASP.NET Core Web API, Entity Framework Core, C#):** Se construye una solución centralizada en C# bajo .NET 10 con controladores RESTful. **ASP.NET Core Web API** expone los endpoints segmentados para clientes y administradores, mientras que **Entity Framework Core** actúa como ORM gestionando el contexto de datos (DbContext) y las transacciones de lectura/escritura contra la base de datos.
- **Base de Datos (Microsoft SQL Server):** Motor de base de datos relacional donde reside el esquema de tablas maestras, transaccionales y de auditoría (Usuarios, Productos, Categorías, Promociones, Pedidos, DetallesPedido). Administra la integridad referencial mediante claves primarias, foráneas y restricciones CHECK.
- **Seguridad (Identity & BCrypt.NET):** Se implementa autenticación mediante **Identity** para proteger las rutas administrativas (/api/admin/\*) mediante el atributo [Authorize] en los controladores. Las contraseñas de los usuarios administradores se almacenan cifradas en base de datos utilizando **BCrypt.NET** para mitigar riesgos de filtración.
- **Diseño & Prototipado (Figma):** Herramienta utilizada para elaborar los mockups interactivos y la guía de estilos (paleta de colores, tipografía y

componentes UI) de ambos frontends antes de la codificación, garantizando consistencia visual.

- **Gestión de Proyecto (Trello — Kanban/Scrum):** Plataforma de seguimiento colaborativo donde se organizan los sprints, backlog, historias de usuario y tareas del equipo en columnas (Lista de Tareas, Por hacer, En progreso, Revisión/QA, Completado).
- **Control de Versiones (Git / GitHub):** Sistema de control de versiones distribuido para la integración del código fuente. Se gestiona un flujo de trabajo con ramas individuales por cada integrante para respaldar las contribuciones y registrar los avances en el repositorio público.

## 8. Licenciamiento

### **Licencia Seleccionada: Creative Commons Atribución - No Comercial - Compartir Igual 4.0 Internacional (CC BY-NC-SA 4.0)**

- **Justificación:** Al tratarse de un desarrollo de software empresarial con fines académicos elaborado para un emprendimiento real salvadoreño, esta licencia permite compartir, adaptar y construir sobre el código fuente generado siempre que:
  1. Se reconozca la autoría de los estudiantes desarrolladores y de la institución (UDB).
  2. No se use el código base con fines comerciales directos por terceros ajenos sin autorización.
  3. Cualquier trabajo derivado mantenga la misma licencia libre y abierta.

## 9. Bibliografía

Claytonsiemens. (s. f.). *Estilo de arquitectura de n niveles – Azure Architecture Center*. Microsoft Learn. <https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/n-tier>

Wadepickett. (s. f.). *Introducción a Identity en ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/es-es/aspnet/core/security/authentication/identity?view=aspnetcore-10.0&tabs=visual-studio>

(Cartagena, 2026)

Mg. Emerson Cartagena (UDB) Guía 10: Autenticación y Autorización en .NET Core con Identity

*Installing with Vite - Installation.* (s. f.). Tailwind

CSS. <https://tailwindcss.com/docs/installation/using-vite>

*Cross-Origin Resource Sharing (CORS) - HTTP | MDN.* (2026, 4

septiembre). <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>

Wadepickett. (s. f.-b). *Tutorial: Creación de una API web basada en controlador con ASP.NET Core.* Microsoft

Learn. <https://learn.microsoft.com/es-es/aspnet/core/tutorials/first-web-api?view=aspnetcore-10.0&tabs=visual-studio>